

RISING BEYOND BORDERS

Document 12 — Contact, Forms & Interaction Layer
Master Specification + Claude Code Implementation Prompt

Status: Implementation specification

Purpose: Establish a reusable, production-ready interaction architecture for contact and inquiry experiences without inventing RBB's contact details, form recipients, backend provider, email service, submission policy or data-retention practices.

1. Why This Phase

Document 08 established the Contact & Organization information architecture but intentionally kept forms and contact details pending because no verified RBB contact information or submission workflow had been supplied. Document 11 similarly keeps donation processing closed. This phase creates the interaction layer needed for future approved forms while preserving the current no-fake-submission behavior.

2. Scope

- Contact page interaction architecture.
- Reusable inquiry form component architecture.
- Get Involved inquiry/application patterns for Volunteer and Partner pages.
- Newsletter interaction architecture, if RBB later supplies an approved provider and consent workflow.
- Client-side validation and accessible error/success states.
- Submission-state model that can support a real backend later.
- Anti-spam and security integration points without selecting an unapproved provider.
- Privacy/consent architecture without inventing a privacy policy.
- Shared field and form primitives to prevent duplicated implementations.

3. Current Routes & Intended Interaction

Route	Current / intended interaction
/contact	General contact/inquiry experience; actual recipient and submission channel remain pending.
/get-involved/volunteer	Volunteer interest/application architecture; no invented eligibility or application process.
/get-involved/partner	Partnership inquiry architecture; no invented packages or recipient.
/get-involved/fundraise	Fundraising interest architecture only if RBB approves a submission path.
/get-involved/donate	Donation page remains informational/pending until Document 11 payment decisions are supplied.

4. Contact Interaction Model

4.1 Contact Methods

Render only verified RBB contact methods from the existing shared contact source. Do not manufacture email addresses, phone numbers, office addresses, hours or social links.

4.2 General Inquiry

Provide a reusable inquiry form shell only when a verified submission destination exists. Until then, show the existing approved pending-contact state.

4.3 Get Involved Routing

Users should be directed to the most relevant existing path—Volunteer, Partner, Fundraise or Donate—rather than forcing every inquiry through a generic form.

4.4 Careers

Keep careers informational unless RBB supplies an approved job/application workflow.

5. Reusable Form Architecture

Suggested components: FormShell, FormField, TextInput, Textarea, SelectField, CheckboxField, FieldError, FormSummary, SubmitButton, SubmissionStatus.

Suggested shared model: formId, status, fields, recipient, provider, endpoint, consent, antiSpam, successMessage, errorMessage, privacyLink.

The form definition should be content/config driven where practical, while validation and submission logic remain reusable.

6. Submission State Machine

State	Behavior
disabled	No form fields are presented; show an approved informational/pending state.
ready	Form is available with approved fields and a verified submission configuration.
submitting	Prevent duplicate submission and communicate progress accessibly.
success	Show only approved success copy; do not promise response times unless supplied.
error	Explain that submission did not complete and provide an approved alternative contact path if available.

7. Field & Validation Rules

- Only collect information that is necessary for the stated inquiry.
- Required fields must be explicitly labelled and programmatically associated with their inputs.
- Errors must identify the field and explain how to correct it.
- Do not reject users based on invented eligibility criteria.
- Do not collect sensitive information unless RBB explicitly requires it and the legal/privacy basis is approved.
- Client-side validation is for user experience; a future backend must validate again.
- Do not place recipient email addresses, API keys, provider secrets or other credentials in client-side code.

8. Consent, Privacy & Data Handling

No RBB privacy policy, retention schedule, legal basis, data processor or consent language has been supplied in the current source set. Therefore the implementation must provide integration points without inventing policy text. A

consent checkbox should only be rendered when RBB supplies the exact purpose and approved wording. A privacy-policy link should only appear when an approved policy route exists.

9. Anti-Spam & Security Architecture

- Create an anti-spam integration point rather than selecting a provider without approval.
- A future backend must rate-limit and validate submissions server-side.
- Do not rely on hidden client-side fields as the sole spam defense.
- Do not expose API keys or email-service credentials in the frontend bundle.
- Avoid echoing submitted personal data into URLs, analytics events or error messages.
- Prevent duplicate submissions and handle network retries safely.

10. Newsletter

Earlier phases removed fake newsletter success behavior because no approved newsletter provider or workflow exists. Keep newsletter UI in a pending state until RBB supplies the provider, signup destination, consent wording, confirmation process and unsubscribe/privacy requirements. Do not silently store email addresses in the website frontend.

11. UX & Accessibility

- One clear form heading and logical field order.
- Visible labels; placeholders must not replace labels.
- Keyboard-accessible controls and visible focus states.
- Errors announced appropriately to assistive technologies.
- Submission status communicated without relying on color alone.
- Loading state must prevent accidental duplicate submission.
- Mobile layouts must work at 320px and above without horizontal scrolling.
- Respect reduced-motion preferences.

12. Out of Scope

- Choosing a form/email provider.
- Choosing a CRM.
- Creating a backend API.
- Selecting a CAPTCHA/anti-spam vendor.
- Creating a privacy policy or legal terms.
- Creating automated email templates or response-time promises.
- Adding real donation/payment processing.
- Inventing contact recipients or organizational email addresses.

13. Claude Code Implementation Prompt

You are implementing Document 12 – Contact, Forms & Interaction Layer for the Rising Beyond Borders website.

CONTEXT

- Documents 01-11 are implemented and approved.
- RBB is an inclusive NGO.
- Document 08 created the Contact & Organization architecture but intentionally kept actual contact details/forms pending.
- Document 11 keeps donations closed until verified payment configuration exists.
- Do not invent RBB contact details, recipients, providers, policies or backend workflows.

GOAL

Build a reusable interaction/form architecture that is safe to activate when RBB supplies verified configuration.

ROUTES

- /contact
- /get-involved/volunteer
- /get-involved/partner
- /get-involved/fundraise
- Do not turn /get-involved/donate into a form or payment flow.

CONTACT

Use the existing shared contact content source.

Only verified contact methods may render.

If no verified submission destination exists, keep the existing pending/informational state.

FORM COMPONENTS

Create/reuse reusable primitives such as:

- FormShell
- FormField
- TextInput
- Textarea
- SelectField
- CheckboxField
- FieldError
- FormSummary
- SubmitButton
- SubmissionStatus

DATA MODEL

A shared form configuration may contain:

```
{
  formId,
  status,
  fields,
  recipient,
  provider,
  endpoint,
  consent,
  antiSpam,
  successMessage,
  errorMessage,
  privacyLink
}
```

Only approved/ready configurations may render active submission forms.

STATE

disabled -> informational/pending state

ready -> active form

submitting -> progress + duplicate-submit protection

success -> approved success copy

error -> clear failure state + approved alternative if available

CRITICAL RULES

- Do not invent email addresses, phone numbers or recipients.
- Do not invent response times.
- Do not invent eligibility requirements.
- Do not invent privacy policy language.
- Do not invent consent language.
- Do not choose a form/email/CRM/anti-spam provider.
- Do not expose API keys or provider secrets.
- Do not build a fake form that appears to submit.

- If no verified endpoint exists, keep the form disabled.

VALIDATION

- Accessible labels and errors.
- Client-side validation only for UX; future backend must validate again.
- No sensitive data in URLs or analytics.
- Prevent duplicate submissions.
- Keyboard/focus/reduced-motion support.
- Responsive at 320px+.

NEWSLETTER

Keep newsletter interaction pending unless a verified provider, consent, confirmation and unsubscribe/privacy process is supplied.

SECURITY

Create integration points for server-side validation, rate limiting and anti-spam. Do not implement or select an unapproved provider.

INTEGRATION

- Preserve existing Contact, Volunteer, Partner and Fundraise content.
- Reuse shared components/data.
- Do not change donation processing.
- Do not create a new backend.
- Do not alter earlier approved information architecture unnecessarily.

VALIDATION

1. Production build passes with no warnings.
2. Test all relevant routes at 320, 375, 768, 1024, 1280 and 1440px.
3. No horizontal overflow.
4. No console errors/warnings.
5. No fake submission success.
6. Disabled forms cannot submit.
7. No credentials/secrets in client code.
8. Existing redirects and routes remain intact.
9. Accessibility checks pass.
10. Scan production content for invented contact details, providers, privacy claims, response times and eligibility rules.

Do not commit or stage changes unless explicitly requested.

REPORT

Return:

- files created/modified
- form architecture
- route behavior
- current enabled/disabled states
- validation/accessibility results
- security considerations
- any RBB approval decisions required.

14. Approval Checklist

- ✓ Contact interaction architecture is reusable.
- ✓ No invented contact details or recipients.
- ✓ No unapproved form/provider/backend selected.
- ✓ Disabled state is clear where configuration is missing.
- ✓ Form validation and error states are accessible.
- ✓ Submission states prevent duplicate actions.
- ✓ No credentials or secrets are exposed.
- ✓ Newsletter remains pending without an approved workflow.
- ✓ Donation/payment behavior is unchanged.
- ✓ Responsive/accessibility/build/console checks pass.
- ✓ Earlier approved routes remain intact.

Next prerequisite: Once RBB supplies verified contact/form configuration, a later implementation phase can activate the appropriate submission provider and backend. This document intentionally does not authorize that activation.